Lab 4 AVCV 2026/2027: Fire-risk classification from satellite images

CUSTOM and FINE-TUNING networks on the FireRisk subset

Daniel Truzman (100475161) César Álvarez-Cascos (100475191) • CodaBench team `100475161_100475191`

1. INTRODUCTION

Task. Given one RGB satellite image, predict one of seven FireRisk classes: Water, Very Low, Low, Moderate, High, Very High and Non-burnable. This is *image classification*: the label describes the risk of the area, not the presence of fire. Two properties make it hard:

- **Class imbalance:** Water is 2.4% of the images, Very Low 31%.
- **Ordinal, similar classes:** neighbouring risk levels (Low / Moderate / High) look alike.

Dataset. A FireRisk subset [1]: 5,861 images of 320×320 px, split (stratified) into 3,516 train, 1,172 validation and 1,173 test images with hidden labels (Table 3). Both final networks take 224×224 crops as input.

Scope. We built two systems:

- **CUSTOM:** a VGG-style CNN trained from scratch, tuned with a staged random search.
- **FINE-TUNING (FT):** ImageNet-pretrained torchvision models; Swin-T was selected.

We studied width and depth, training length, class weighting, augmentation and mixing, ensembles and retraining on train+val.

2. METHODOLOGY

2.1. Preprocessing and data augmentation

Pipeline. Images are read as floats in $[0, 1]$ and normalised with the ImageNet mean and standard deviation (required by the pretrained weights; kept for CUSTOM so both share one pipeline).

- **Train:** random 224×224 crop of the 320×320 image, then GPU batch augmentation: horizontal/vertical flips, 90° rotations and random gamma, brightness, contrast and saturation (*light* or *strong* preset).
- **Validation and test:** centre 224×224 crop, no augmentation, no test-time augmentation (TTA).

Why. Satellite images have no canonical orientation, so flips and rotations add samples without changing the label; colour changes stay mild to keep vegetation cues. The search chose *strong* for CUSTOM and *light* for Swin-T. Mixup [2], CutMix [3], TTA and evaluating at 320 px gave no gain beyond seed noise (±0.008; Table 2).

2.2. System design: CUSTOM network

Architecture (Table 5). Four VGG-style blocks [4], each [Conv 3×3 – BatchNorm [5] – ReLU]×2 + MaxPool 2×2, with 48–96–192–384 channels. Global average pooling (GAP) [6] feeds FC(384,256) – ReLU – Dropout(0.3) – FC(256,7). Only primitive layers are used; 2.74 M parameters.

Changes from the reference (three blocks of one 5×5 conv with 6–16–32 channels, no normalisation, Flatten head), each kept only if it raised validation macro-F1:

- **GAP instead of Flatten:** the Flatten FC layer held ∼99% of the parameters and scored lower.
- **Two 3×3 convs + BatchNorm per block:** same receptive field as 5×5, fewer weights, stabler training.
- **Wider (48 base channels):** +0.03. Going further (64 channels, 5 blocks, bigger crops) did not help.
- **GAP as a plain mean:** `AdaptiveAvgPool2d` has a non-deterministic CUDA backward.

2.3. System design: FINE-TUNING network

Pretrained model. Instead of the provided AlexNet [7] we compared three modern backbones (24–28 M parameters) with torchvision ImageNet-1k weights [8]: ConvNeXt-T [9], Swin-T [10] and EfficientNetV2-S [11]. EfficientNetV2-S failed (non-finite loss in fp16) and was replaced by ResNet-50 [12]. In every model the classifier was replaced by a new `Linear(in, 7)` (768→7 for Swin-T) and adaptive pooling by a deterministic mean.

Adaptation.

- **No frozen layers:** backbone and head are trained together.
- **Matching preprocessing:** 224×224 input and ImageNet normalisation, as in pretraining.
- **Two learning rates** [13]: a small one for the backbone and the head rate ×1 or ×10; no weight decay on biases and norms; stochastic depth [14].

Selected: Swin-T (backbone LR $2.37 \cdot 10^{-5}$, head ×1, drop path 0.2): best validation macro-F1 (0.546) and the most stable run (train F1 0.77, vs 0.99 for ResNet-50).

2.4. Training protocol and model selection

Table 1: Training configuration of the selected models.

	CUSTOM (n07_60ep)	FT (Swin-T, swin_p2_1)
Loss	Cross-entropy, class weights $\propto 1/\sqrt{n_c}$, no label smoothing, no mixup	
Optimiser & schedule	AdamW [15]; linear warm-up + cosine decay [16]; fp16 mixed precision	
Learning rate	$3.17 \cdot 10^{-4}$	backbone $2.37 \cdot 10^{-5}$, head $\times 1$
Weight decay	0.0464	0.0395
Batch / warm-up	128 / 1 epoch	32 / 2 epochs
Epochs (selected epoch)	60 (36)	30 (26)
Augmentation	strong	light, drop path 0.2
Common	Seed 42; full train split (no subsampling); Kaggle Tesla T4; PyTorch 2.11 [17]	

Search. Random search [18] with median-stopping pruning [19], from coarse to fine:

- **CUSTOM, phases 1–3:** 24 architecture/optimiser trials, 16 trials of secondary settings, 3 finalists at 40 epochs. All plateaued at 0.45–0.465 with train $\approx$ val (underfitting).
- **CUSTOM, phases 4–5:** targeted changes; widening and 60 epochs were the only clear gains. A larger 121-trial search reached 0.524 on validation but 0.47 on public, so it was discarded.
- **FT:** 10 configurations per network (10 epochs), then the best 2 per network retrained for 30 epochs.

Selection and reproducibility. We keep the weights of the epoch with the best validation macro-F1; there is no early stopping in the final runs. Training is fully deterministic (`use_deterministic_algorithms`, fixed seeds for all libraries and DataLoader workers): training one epoch twice gives bit-identical weights.

3. RESULTS AND ERROR ANALYSIS

Metrics. The main metric is macro-F1 (mean of the seven per-class F1), so rare classes count as much as frequent ones. Validation F1 drove every decision; training F1 is only a diagnostic; the public CodaBench score was never used for selection.

What helped and what did not (Table 2):

- **CUSTOM:** the reference reached ∼0.36 and our first GAP version 0.440, both underfitting. The search plateaued at 0.45–0.465. More capacity (48 channels, 2 convs/block) and 60 epochs gave the selected model: **0.502**.
- **No gain:** inverse-frequency weights (worse), 64 channels, 5 blocks, larger crops, CutMix, 80 epochs, ensembles, per-class threshold tuning and retraining on train+val. Seeds alone move results by ±0.008.
- **FT:** every modern backbone beat AlexNet (0.452); Swin-T was best with **0.546**.

Curves and confusion matrices. For CUSTOM (Figure 2) train and validation curves stay together and validation F1 saturates at 0.49–0.50 after epoch 30: no overfitting, the model is limited by the data. Swin-T fits the training set better (train F1 0.77 vs 0.546 on validation). Figure 3 shows the validation confusion matrices (row-normalised %) [TODO: describe once generated].

Error analysis (Table 3, Figure 4).

- **Easy:** Water (0.85) and Non-burnable (0.80).
- **Hard:** the intermediate levels Low (0.22), Moderate (0.32) and High (0.33) for CUSTOM, and Low, Moderate and Very High for FT, which points to confusions between neighbouring risk levels [TODO: confirm with Figures 3 and 4].
- **Limited signal:** even on its own training images (no augmentation) CUSTOM only reaches 0.53 (Low 0.30), so the problem is the information in the image, not model capacity.
- **CUSTOM vs FT:** CUSTOM is small (2.7 M parameters), fast and stable; FT gains +0.04 on validation at a higher compute cost.

Competition (Table 4). Best public scores: **0.49** CUSTOM (p4_09_B, submission 1036062) and **0.48** FT (swin_p2_1). Every model loses about 0.06 from validation to public, because the same validation images were reused for hundreds of trials and the public subset is small. For example, the 121-trial model had the best validation score (0.524) but only 0.47 on public.

4. CONCLUSIONS

- **Largest effects.** CUSTOM: GAP head, more width and a longer schedule (0.44 → 0.50). FT: the ImageNet pretraining itself, with a small backbone learning rate (0.546 in 30 epochs).
- **Scratch vs fine-tuning.** FT is clearly better on validation, but both score about the same on the public subset (0.49 vs 0.48).
- **Main limitation.** The intermediate classes are hard even on training images, and reusing one validation set for selection inflates its scores (∼0.06). The original FireRisk paper reports only 61–65% accuracy with ∼91k images [1].
- **Next steps.** k-fold cross-validation for honest selection, losses that use the class order, remote-sensing

pretraining and higher input resolution.

Individual contributions.

- **Daniel Truzman (100475161):** CUSTOM phases 3–5 (widening to 48×2, 60 epochs, selection of n07_60ep), deterministic Kaggle setup, CutMix / ensemble / threshold / train+val experiments, notebook cleanup and submissions. [TODO: confirm; GPU pipeline/AMP: Dani or César?]
- **César Álvarez-Cascos (100475191):** notebook Parts 1–2 (dataset, transforms), two-machine random search (phases 1–2), 121-trial deep search and the fine-tuning search (Swin-T). [TODO: confirm]

Table 2: Main experiments: validation macro-F1 at the best epoch (L10: mean of the last 10 epochs). Reference and selected models in bold.

Cat.	Change	Objective	Budget	Val F1
CUSTOM	**Reference notebook (StepLR)**	Baseline	[TODO:] ep	≈0.36
CUSTOM	GAP instead of Flatten (first version)	Fewer FC parameters; train F1 ≈0.42	[TODO:] ep	0.440
CUSTOM	Phases 1–3 random search	Architecture, optimiser, regularisation	15 ep (finalists 40)	0.45–0.465
CUSTOM	Inverse-frequency class weights	Help minority classes	[TODO:] ep	L10 0.431 vs 0.453
CUSTOM	48 ch, 2 convs/block (p4_09_B)	More capacity (underfitting)	40 ep	0.496 (L10 0.483)
CUSTOM	64 ch; 5 blocks; crop 288/eval 320	More capacity / resolution	40 ep	no gain
CUSTOM	**60 epochs (n07_60ep)**	Longer schedule	60 ep	**0.502** (L10 0.489)
CUSTOM	CutMix (standard; risk-only)	Regularisation	[TODO:]; 80 ep	L10 0.483; 0.470 vs 0.486
CUSTOM	Deep search (F3_B_038)	Joint search, 121 trials	80 ep, input 320	0.524
FT	**AlexNet (provided recipe)**	Baseline	5 ep	0.452
FT	ConvNeXt-T (cnx_p2_0)	Modern CNN backbone	30 ep	0.505
FT	ResNet-50 (r50_p2_0)	Residual backbone (train F1 0.99)	30 ep	0.539
FT	**Swin-T (swin_p2_1)**	Transformer backbone	30 ep	**0.546**

Table 3: Class counts and per-class validation F1 of the selected models. [TODO: P/R]

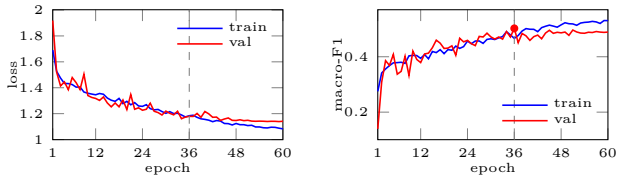
Class	Train	Val	CUSTOM	FT
0 Water	86	29	0.847	[TODO:]
1 Very Low	1,088	363	0.598	
2 Low	535	178	0.221	
3 Moderate	431	144	0.322	
4 High	315	105	0.333	
5 Very High	163	54	0.400	
6 Non-burnable	898	299	0.796	
Total / macro	3,516	1,172	0.502	0.546

Table 4: Public CodaBench macro-F1 (fixed undisclosed test subset). Also: F1_B_037 0.47, swin_p2_0 0.48, r50_p2_0 0.47.

ID	Date	Models (custom / ft)	CUSTOM	FT
1036062	05 Nov	p4_09_B / placeholder	0.49	–
[TODO:]	[TODO:]	n07 (train+val) / [TODO:]	0.494	[TODO:]
[TODO:]	[TODO:]	F3_B_038 / AlexNet	0.47	[TODO:]
[TODO:]	[TODO:]	[TODO:] / swin_p2_1	[TODO:]	0.48
[TODO:]	[TODO:]	[TODO:] / FT ensemble (3)	[TODO:]	0.48

Table 5: Final CUSTOM architecture.

Stage	Layers	Output
Input	RGB crop	3 × 224 × 224
Blocks 1–4	[Conv3×3-BN-ReLU]×2, MaxPool 2×2 (spatial size 112/56/28/14)	48/96/192/384 ch 384 × 14 × 14
GAP	spatial mean	384
Head	FC 384→256, ReLU, Dropout 0.3, FC 256→7	7
2,739,447 parameters; Kaiming initialisation.		



(a) Loss **(b)** Macro-F1

Figure 2: Train (augmented) and validation curves, CUSTOM n07_60ep; dashed: selected epoch 36. [TODO: FT curves]

[TODO: CUSTOM CM]

[TODO: FT CM]

(a) CUSTOM (n07_60ep) **(b)** FT (swin_p2_1)

Figure 3: Validation confusion matrices, row-normalised (%); rows = true, columns = predicted class.

[TODO: one row of misclassified validation images, each with true / predicted label]

Figure 4: Representative validation errors of the selected models (true → predicted class).

REFERENCES

[1] Shuchang Shen, Sachith Seneviratne, Xinye Wanyan, et al. FireRisk: A remote sensing dataset for fire risk assessment with benchmarks using supervised and self-supervised learning. In *DICTA*, 2023.

[2] Hongyi Zhang, Moustapha Cisse, Yann N. Dauphin, et al. mixup: Beyond empirical risk minimization. In *ICLR*, 2018.

[3] Sangdoo Yun, Dongyoon Han, Seong Joon Oh, et al. CutMix: Regularization strategy to train strong classifiers with localizable features. In *ICCV*, 2019.

[4] Karen Simonyan and Andrew Zisserman. Very deep convolutional networks for large-scale image recognition. In *ICLR*, 2015.

[5] Sergey Ioffe and Christian Szegedy. Batch normalization: Accelerating deep network training by reducing internal covariate shift. In *ICML*, 2015.

[6] Min Lin, Qiang Chen, and Shuicheng Yan. Network in network. In *ICLR*, 2014.

[7] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E. Hinton. ImageNet classification with deep convolutional neural networks. In *NeurIPS*, 2012.

[8] Olga Russakovsky, Jia Deng, Hao Su, et al. ImageNet large scale visual recognition challenge. *IJCV*, 2015.

[9] Zhuang Liu, Hanzi Mao, Chao-Yuan Wu, et al. A ConvNet for the 2020s. In *CVPR*, 2022.

[10] Ze Liu, Yutong Lin, Yue Cao, et al. Swin transformer: Hierarchical vision transformer using shifted windows. In *ICCV*, 2021.

[11] Mingxing Tan and Quoc V. Le. EfficientNetV2: Smaller models and faster training. In *ICML*, 2021.

[12] Kaiming He, Xiangyu Zhang, Shaoqing Ren, et al. Deep residual learning for image recognition. In *CVPR*, 2016.

[13] Jeremy Howard and Sebastian Ruder. Universal language model fine-tuning for text classification. In *ACL*, 2018.

[14] Gao Huang, Yu Sun, Zhuang Liu, et al. Deep networks with stochastic depth. In *ECCV*, 2016.

[15] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *ICLR*, 2019.

[16] Ilya Loshchilov and Frank Hutter. SGDR: Stochastic gradient descent with warm restarts. In *ICLR*, 2017.

[17] Adam Paszke, Sam Gross, Francisco Massa, et al. PyTorch: An imperative style, high-performance deep learning library. In *NeurIPS*, 2019.

[18] James Bergstra and Yoshua Bengio. Random search for hyper-parameter optimization. *JMLR*, 2012.

[19] Daniel Golovin, Benjamin Solnik, Subhodeep Moitra, et al. Google vizier: A service for black-box optimization. In *KDD*, 2017.